

<HLS Programming with FPGAs>

The HLS Book

Chapter 3

CORDIC 2 and Fixed-Point Representation

Choi, Young-kyu

Recap: CORDIC Synthesis Result

```
// for (int j = 0; j < NUM_ITERATIONS; j++) {  
  // Determine if we are rotating by a positive or negative angle  
  int sigma = (theta < 0) ? -1 : 1;  
  
  // Multiply previous iteration by 2^(-j)  
  COS_SIN_TYPE cos_shift = current_cos * sigma * factor;  
  COS_SIN_TYPE sin_shift = current_sin * sigma * factor;  
  
  // Perform the rotation  
  current_cos = current_cos - sin_shift;  
  current_sin = current_sin + cos_shift;  
  
  // Determine the new theta  
  theta = theta - sigma * cordic_phase[j];  
  factor = factor / 2;  
}
```

Latency Interval	Trip Count	Pipelined
-	80	no
18	15	yes

The II Violation in module 'cordic' (loop 'CORDIC_LOOP'): Unable to enforce a carried dependence constraint (II = ('current_sin_write_ln25', [cordic_float.cpp:25](#)) of variable 'current_sin', [cordic_float.cpp:25](#) on local variable 'current_sin', [cordic_float.cpp:25](#)) on local variable 'current_sin'.

Learning Objectives

- Optimize baseline CORDIC implementation
- Explain fixed-point and floating-point representations
- Compare the hardware cost for using fixed-point and floating-point representation

Synthesizing CORDIC in Integer/Short Types

- Change THETA_TYPE and COS_SIN_TYPE to int/short types and run synthesis again
- Synthesis result:
 - float type: (from last class)

Latency	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
79	316.000	-	80	-	no	0	10	1425	975	0
77	308.000	18	15	5	yes	-	-	-	-	-

- int type:

Latency	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
14	56.000	-	15	-	no	0	6	703	558	0
12	48.000	5	2	5	yes	-	-	-	-	-

- short type:

Latency	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
17	68.000	-	18	-	no	0	2	165	343	0
15	60.000	4	3	5	yes	-	-	-	-	-

- II issue

The II Violation in module 'cordic' (loop 'CORDIC_LOOP'): Unable to enforce a carried dependence constraint (II = 1, distance = 1, offset = 0) of DSP[35] ('current_sin', [cordic_short.cpp:28](#)) and 'mul' operation of DSP[34] ('sin_shift', [cordic_short.cpp:25](#)).

```
for (int j = 0; j < NUM_ITERATIONS; j++) {  
    // Determine if we are rotating by a positive or negative angle  
    int sigma = (theta < 0) ? -1 : 1;  
  
    // Multiply previous iteration by 2^(-j)  
    COS_SIN_TYPE cos_shift = current_cos * sigma * factor;  
    COS_SIN_TYPE sin_shift = current_sin * sigma * factor;  
  
    // Perform the rotation  
    current_cos = current_cos - sin_shift;  
    current_sin = current_sin + cos_shift;  
  
    // Determine the new theta  
    theta = theta - sigma * cordic_phase[j];  
  
    factor = factor / 2;  
}
```

(II=3)

3.6 CORDIC Optimization

```
CORDIC_LOOP:
for (int j = 0; j < NUM_ITERATIONS; j++) {

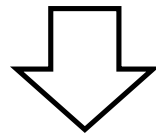
    int sigma = (theta < 0) ? -1 : 1;

    COS_SIN_TYPE cos_shift = current_cos * sigma * factor;
    COS_SIN_TYPE sin_shift = current_sin * sigma * factor;

    current_cos = current_cos - sin_shift;
    current_sin = current_sin + cos_shift;

    theta = theta - sigma * cordic_phase[j];
    factor = factor / 2;
}
```

<Baseline>



Optimization 1: Remove loop-carried dep with right shift

Optimization 2: Replace multiplication with mux/add/sub

```
CORDIC_LOOP:
for (int j = 0; j < NUM_ITERATIONS; j++) {
    COS_SIN_TYPE cos_shift = current_cos >> j;
    COS_SIN_TYPE sin_shift = current_sin >> j;

    current_cos = (theta < 0) ? current_cos + sin_shift : current_cos - sin_shift;
    current_sin = (theta < 0) ? current_sin - cos_shift : current_sin + cos_shift;

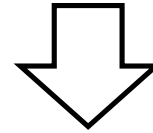
    theta = (theta < 0) ? theta + cordic_phase[j] : theta - cordic_phase[j];
}
```

<Optimized>

- Synthesis result after CORDIC optimizations (short)

Before CORDIC optimization:

Iteration	Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
-	-	18	-	no	0	2	165	343	0
4		3	5	yes	-	-	-	-	-



After CORDIC optimization:

Iteration	Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
-	-	8	-	no	0	0	67	370	0
2		1	5	yes	-	-	-	-	-

Checking Correctness

- Change THETA_TYPE and COS_SIN_TYPE to int/short in your testbench file
- Run C-simulation of your float, integer, short version
- RMSE/maximum error of sine/cosine values:

– float type:

RMSE	Max error
0.0245	0.0627

– int type:

RMSE	Max error
0.707	1.0

– short type:

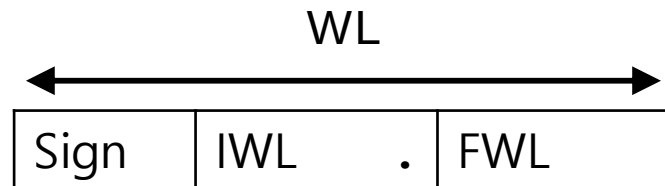
RMSE	Max error
0.707	1.0

} Incorrect output

- Simply changing a variable type from **float** to **int/short** is **NOT** the correct way of adopting fixed-point representation
- So what is fixed-point representation?

Fixed-Point Representation

- A number with a fixed number of digits before/after the radix point
- Format:



S = sign bit

WL = word length

IWL = integer word length

FWL = fractional word length

- Example) $8.5_{10} = 1000.1_2$
 - s=1b, IWL=6b, FWL=1b: 00010001
 - s=1b, IWL=5b, FWL=2b: 00100010
- Negative number: 2's complement

-2^2	2^1	2^0	two's complement
0	1	1	= 3
0	1	0	= 2
0	0	1	= 1
0	0	0	= 0
1	1	1	= -1
1	1	0	= -2
1	0	1	= -3
1	0	0	= -4

(sign bit)

- Converting sign:
 $-1 * a = \sim a + 1$

- Precision issue -> rounding

<FWL = 2 bits>			<FWL = 1 bits>	
0b0100.00	= 4.0		0b0100.0	= 4.0
0b0011.11	= 3.75		0b0011.1	= 3.5
0b0011.10	= 3.5		0b0011.1	= 3.5
0b0011.01	= 3.25		0b0011.0	= 3.0
0b0011.00	= 3.0	Round to	0b0011.0	= 3.0
0b1100.00	= -4.0	→ Negative	0b1100.0	= -4.0
0b1011.11	= -4.25	Infinity	0b1011.1	= -4.5
0b1011.10	= -4.5		0b1011.1	= -4.5
0b1011.01	= -4.75		0b1011.0	= -5.0
0b1011.00	= -5.0		0b1011.0	= -5.0

– cf) round to positive infinity, or round to nearest even

- Fixed-point arithmetic

- Ex) 0 sign bit, IWL=4b, FWL=4b

- $8.0_{10} + 0.1875_{10}$

- $= 1000_0000_2 + 0000_0011_2$

- $= 1000_0011_2$

- $= 8.1875_{10}$

- Ex) 1 sign bit, IWL=7b, FWL=0b

- $-8_{10} + 3_{10}$

- $1111_1000_2 + 0000_0011_2$

- 1111_1011_2

- -5_{10}

$$\begin{array}{l}
 8 : 0000_1000 \\
 \rightarrow -8 : 1111_0111 + 1 = 1111_1000
 \end{array}$$

3.5.5 Arbitrary Precision Integer Representation

- `ap_fixed` class
 - `#include "ap_fixed.h"`
 - `ap_ufixed<WL, IWL> a = 4.0; // unsigned integer`
 - `ap_fixed<WL, IWL> b = -3.0; // signed integer`
- `ap_int` class
 - `#include "ap_int.h"`
 - `ap_uint<WL> a = 4; //WL-bit unsigned integer`
 - `ap_int<WL> b = 3; //WL-bit signed integer`
 - Programmer manages IWL/FWL

Exercise

- If sign=1b, IWL=4b, FWL=3b, compute:

$$-3.75_{10} + 1.125_{10}$$

Write down intermediate equation in the given fixed-point representation:

$$\boxed{????_????_2 + ????_????_2 = ????_????_2} = -2.625_{10}$$

Solution to Exercise

- Q) Compute $-3.75_{10} + 1.125_{10}$, if sign=1b, IWL=4b, FWL=3b
- $3.75_{10} = 11.11_2 = 0001_1110_2$
- $-3.75_{10} = 1110_0001_2 + 1 = 1110_0010_2$
- $1.125_{10} = 1.001_2 = 0000_1001_2$
- $-3.75_{10} + 1.125_{10} = 1110_0010_2 + 0000_1001_2$
 $= 1110_1011_2$
 $= -2.625_{10}$

Floating-Point Representation

- Defined by IEEE standard 754-1985
- Representations
 - Single precision (32-bit) : float
 - Double precision (64-bit) : double
- Format:

1 bit	single: 8 bits double: 11 bits	single: 23 bits double: 52 bits
Sign	Exponent	Significand (Mantissa)

$$\text{value} = (-1)^{\text{Sign}} \times (1 + \text{Significand}) \times 2^{(\text{Exponent} - \text{Bias})}$$

** Bias single: 127, double: 1023

- Advantage: Can represent a large range of real numbers

Floating-point Addition

- Ex) Compute $1.11_2 \times 2^1 + 1.10_2 \times 2^{-1}$ ($= 3.5_{10} + 0.75_{10}$)
 - Step 1) Align binary point
 $= 1.11_2 \times 2^1 + 0.011_2 \times 2^1$
 - Step 2) Add significand
 $= 10.001_2 \times 2^1$
 - Step 3) Normalize result
 $= 1.0001_2 \times 2^2$
 - Step 4) Round (and re-normalize) result
 $= 1.00_2 \times 2^2$
- Several operations needed to perform floating-point addition (very **costly** in terms of **hardware**, speed, and power)

Hardware Cost of Add

- Float (32-bit):

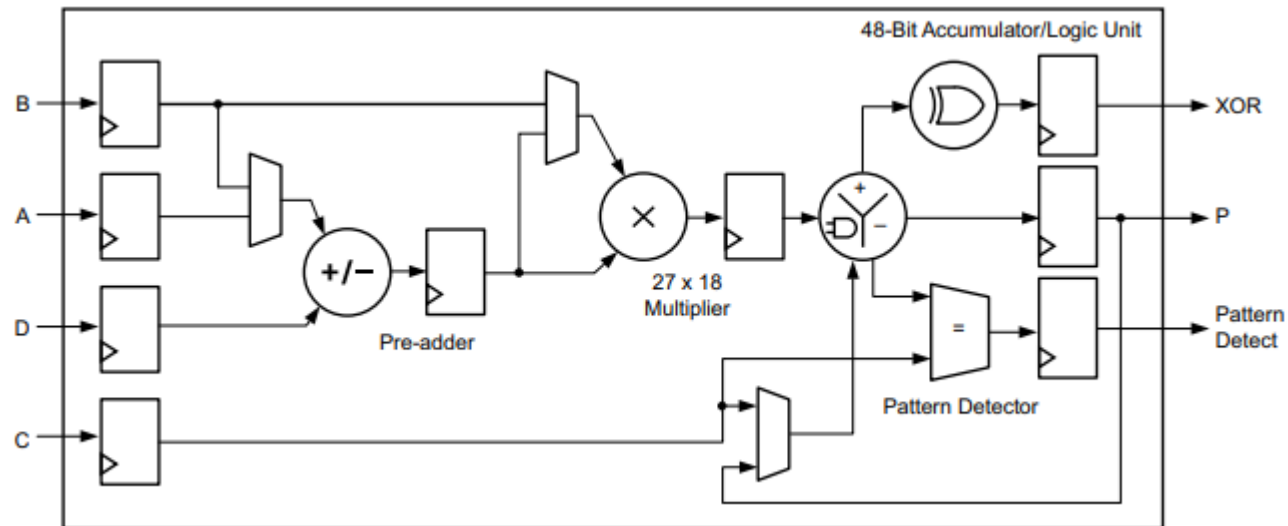
DSP	FF	LUT
2	325	206
- Integer (32-bit):

DSP	FF	LUT
0	0	39
- Short (16-bit):

DSP	FF	LUT
0	0	23

DSP48

- Dedicated multiply-and-accumulate unit
 - 27b * 18b multiplier
 - 27b adder,
 - 48b accumulator
 - Can be used for floating-point operation



Xilinx, UltraScale Architecture DSP Slice User Guide (UG579)

Resource Consumption Comparison

- Synthesis result comparison between float vs int/short (same as previous slide)

– float type:

Latency(	Latency(ns)	Iteration	Latency	Interval	Trip	Count	Pipelined	BRAM	DSP	FF	LUT	URAM
79	316.000	-	80	-	-	no	0	10	1425	975	0	
77	308.000	18	15	5	yes	-	-	-	-	-	-	

– int type:

Latency(	Latency(ns)	Iteration	Latency	Interval	Trip	Count	Pipelined	BRAM	DSP	FF	LUT	URAM
14	56.000	-	15	-	no	0	6 703	558	0			
12	48.000	5	2	5	yes	-	-	-	-	-		

– short type:

Latency	Latency(ns)	Iteration	Latency	Interval	Trip	Count	Pipelined	BRAM	DSP	FF	LUT	URAM
17	68.000	-	18	-	no	0	2 165 343	0				
15	60.000	4	3	5	yes	-	-	-	-	-		

- But naïve conversion from float type to int/short type leads to wrong result

– float type:

RMSE	Max error
0.0245	0.0627

– int/short type:

RMSE	Max error
0.707	1.0

→ Incorrect output

- Reason for the large error: **IWL/FWL of variables not set** -> produces incorrect result
- Next question: How to assign IWL/FWL systematically?

Summary

- Optimized the CORDIC algorithm by removing loop-carried dependency and replacing multiplication with shift & addition
- Simply changing a variable type from float to int/short is not the correct fixed-point conversion process
- Learned fixed-point and floating-point representations and their arithmetic
- Floating-point representation has a larger dynamic range than fixed-point representation, but consumes more resource